Nom: Hebbache, Ichchou
Prénom: Rayane, Nassim

L2 INFORMATIQUE
Date : 13/03/2026

Luminy Checker

1. *Modèle de données*

(a) Tables

3 tables SQL seront utilisées:

i. Une table **user** qui stocke les données de connexion des utilisateurs avec comme colonnes une clé primaire auto-incrémentée id de type INTEGER et deux autres colonnes username et password de type TEXT correspondant au mot de passe stocké sous forme hachée pour des raisons de sécurité

ii. Une table **affluence** qui stocke les différents signalements d'affluence des utilisateurs avec comme colonnes une clé primaire auto-incrémentée id et 4 autres colonnes user_id, une clé étrangère qui est de type INTEGER qui fait la passerelle avec id de **user** (**user**[id]), place_id de type INTEGER, une clé étrangère faisant la passerelle avec id de place (place[id]), une colonne niveau de type TEXT...pour indiquer le niveau de l'affluence (faible, moyen et fort), timestamp de type TIMESTAMP (heure + date) pour indiquer le moment exact où le signalement a été effectué.

iii. Une table **place** qui stocke les différents lieux du campus avec comme colonnes une clé primaire auto-incrémentée id de type INTEGER et une autre colonne name de type TEXT qui correspond au nom du lieu (restaurant universitaire, cafétéria, bibliothèque universitaire, etc.).

(b) Fonction d'accès

i. **login(user, password)**: retourne la clé id associée à l'utilisateur dans la table user si l'utilisateur existe et si le mot de passe correspond à celui enregistré; retourne -1 sinon

ii. **user_exists(user)**: retourne true si l'utilisateur existe et false sinon

iii. **new_user(user, password)**: si l'utilisateur existe déjà retourne -1; sinon ajoute un utilisateur et retourne la clé id associée dans la table user

v. **add_affluence(user_id, place_id, level)**: ajoute l'affluence d'un lieu par un utilisateur à la base de données

vi. **get_affluence(place_id)**: retourne la liste des derniers signalements effectués par les utilisateurs pour un lieu donné, triés par date

2. Vues

(a) Un en-tête **header** qui sera présent sur toutes les pages du site lorsqu'on est connecté pour permettre à l'utilisateur de naviguer entre les différentes possibilités de notre application web si l'utilisateur est connecté tel qu'une liste des lieux du campus pour voir où signaler de l'affluence, les paramètres de l'application (passer en dark theme, …), un bouton se déconnecter. En revanche si l'utilisateur n'est pas connecté alors il aura la possibilité de se connecter ou créer un compte.
(b) Un bas de page **footer**, utilisé sur toutes les pages
(c) Une page d'accueil **index**, affichant un message de bienvenue lorsque l'on vient de se connecter ou une mini présentation de l'application sinon
(d) Une page de connexion **login**, contenant un formulaire pour se connecter à l'aide de son nom d'utilisateur et de son mot de passe
(e) Une page de création de compte **register**, affichant un formulaire permettant de choisir un nom d'utilisateur et un mot de passe avec un message pour signaler que le nom d'utilisateur choisi est déjà pris
(f) Une page **affluence** qui affiche la liste des différents lieux du campus avec pour chacun leur niveau d'affluence et un bouton permettant à l'utilisateur d'indiquer le niveau d'affluence actuel d'un lieu (redirige vers signalement)
(g) Une page **signalement** permettant à l'utilisateur d'indiquer le niveau d'affluence actuel d'un lieu via un formulaire

3. Routes et logique de contrôle

(a) Une route **GET /** auquel on affichera la page **index**
(b) Les routes **GET /login** et **GET /new_user afficherons** respectivement les pages de connexion (**login**) et de création de compte (**register**)
(c) Une route **POST /login** qui connectera l'utilisateur à partir de l'identifiant et du mot de passe qu'il entrera et sera redirigé vers son profil personnel si la connexion réussit (si l'identifiant et le mot de passe correspondent à un utilisateur enregistré dans la base de donné) vers **GET /login** sinon
(d) Une route **POST /new_user** qui créera un nouveau compte avec l'identifiant et le mot de passe fournis par l'utilisateur et le redirigera vers la page **index** si la création réussit (si le nom de l'utilisateur n'est pas déjà pris) et vers la route **GET /new_user** sinon, avec le message « Cet utilisateur existe déjà ».
(e) Une route **GET /logout** qui déconnectera l'utilisateur et le redirigera vers la route **GET /**
(f) Une route **Get /affluence** qui redirigera vers la vue **affluence** (fait appel à la fonction d'accès **get_affluence(place_id)**)
(g) Une route **POST /signaler** traite le formulaire de signalement d'affluence (fait appel à la fonction d'accès **add_affluence(user_id, place_id, level)**)
(h) Une route **POST /update_user** qui permet à un utilisateur connecté de modifier ses informations personnelles (nom d'utilisateur et/ou mot de passe). Les nouvelles données sont enregistrées dans la base de données après vérification.

(i) Une route **POST /delete_user** qui permet à un utilisateur de supprimer son compte. Cette action entraîne la suppression de ses données dans la base de données et le déconnecte automatiquement de l'application.
(j) Une route **GET /export_data** qui permet à un utilisateur connecté de récupérer ses données personnelles au format JSON.

Comme feature on pourra implémenter des comptes administrateur qui pourront empêcher certains utilisateurs d'effectuer des signalements s'ils fournissent régulièrement de faux signalements.